

Lagos Emergency Intelligence Network

NacosCuab Hackathon '26 · Open Innovation Track

Full Project Breakdown — Phases, Architecture & 48-Hour Build Plan

Executive Summary

LEIN is an AI-powered emergency dispatch and triage coordination system purpose-built for Lagos. When a citizen faces a medical emergency, fire, or security incident, LEIN classifies the event using NLP, scores its severity with a predictive model, and routes the nearest available responder — all in under three seconds. Built in 48 hours on a FastAPI + React stack, LEIN targets six judging dimensions simultaneously: innovation, deep AI integration, technical execution, measurable impact, clean UX, and airtight documentation.

PROJECTED SCORE BREAKDOWN

CRITERIA	WEIGHT	LEIN ADVANTAGE
Innovation & Creativity	20 pts	First emergency coordination tool localised for Lagos geography, Pidgin NLP, and road topology
AI Integration & Relevance	20 pts	4 distinct AI layers: NLP classifier, severity predictor, route optimiser, incident forecaster

Technical Execution	20 pts	FastAPI + React + SQLite, production-grade endpoint design, clean repo, typed schemas
Impact & Problem Relevance	20 pts	Direct life-safety application; every judge in Lagos has experienced the coordination gap
User Experience & Design	10 pts	Two role-based interfaces, real-time Leaflet map, accessible colour system
Presentation & Documentation	10 pts	60-second live demo script, full README, architecture diagram, setup guide

Target total: **92 – 96 / 100**. Even a conservative build delivers well above the 60-point Innovation Grant threshold.

The Problem

Lagos handles over 20 million residents across 20 LGAs. Emergency response coordination is fragmented — there is no unified system that tells a dispatcher which hospital has bed capacity, which responder is closest, or which corridor is currently passable. The result:

- Ambulances arrive at full hospitals and must re-route in real time
- Dispatchers use WhatsApp and phone calls to coordinate, losing minutes
- No historical data exists to predict peak-hour incident clusters
- Language barriers slow intake — most callers speak Pidgin, not formal English

The Hackathon Angle: You are not being judged on whether LEIN is deployed. You are being judged on whether it demonstrates a credible, technically sound, AI-driven solution to a real problem. LEIN does both.

System Architecture

COMPONENT OVERVIEW

Citizen Interface	React SPA — text or voice emergency report, auto-language detection, one-tap severity escalation. Works on mobile browser.
Dispatch Dashboard	React SPA (role-gated) — real-time incident map (Leaflet.js), AI-ranked queue, nearest-hospital panel, responder assignment controls.
Analytics Panel	Incident heatmap by LGA, response-time trend chart, peak-hour forecasting table. Recharts library.
FastAPI Backend	REST API with 8 endpoints: report, classify, assign, resolve, hospitals/nearby, responders/available, stats/heatmap, forecast. Pydantic schemas, async handlers.
AI Pipeline	4 models wired in sequence: (1) NLP Classifier, (2) Severity Scorer, (3) Route Optimiser, (4) Incident Forecaster. See Phase 2.
Database	SQLite for hackathon (schema migrates to PostgreSQL). Tables: incidents, hospitals, responders, assignments, audit_log.
Seed Data	30 Lagos hospitals with coordinates, bed-capacity estimates, and specialisation tags. 10 simulated responder units across 5 LGAs.

48-Hour Build Plan — Phase Breakdown

PHASE 0 | SETUP & SCAFFOLDING [HOUR 0 – 2]

RESPONSIBLE: FULL TEAM

The first two hours are infrastructure. Nothing gets built on a shaky foundation. Every team member must be unblocked before coding starts.

Repo	Accept GitHub org invitation. Confirm branch protection: main protected, all work on feature branches, PRs required to merge.
Backend scaffold	fastapi, uvicorn, sqlalchemy, pydantic, python-dotenv installed. main.py boots on port 8000. /health endpoint returns 200.
Frontend scaffold	create-react-app (or Vite) on port 3000. Axios installed. Proxy set to localhost:8000 in package.json.
Database	SQLite file created. SQLAlchemy models defined for all 5 tables. Alembic migration runs clean.
Seed script	seed.py populates 30 hospitals, 10 responders. Confirmed via /hospitals/nearby?lat=6.52&lng=-3.38 returning data.
AI env	scikit-learn, transformers (or openai SDK), numpy, pandas installed. Test import runs without errors.
README skeleton	Title, team info, problem statement, setup steps (placeholder). Commit to main. Repo now meets minimum submission requirements.

PHASE 1 | CORE BACKEND & DATA LAYER [HOUR 2 – 10]

RESPONSIBLE: BACKEND ENGINEER

Build all 8 API endpoints with mock AI responses first. This unblocks the frontend team immediately. AI models slot in during Phase 2.

POST /report	Accepts: type (str), description (str), lat (float), lng (float), severity_hint (int 1-5). Returns: incident_id, status='pending'.
POST /classify	Accepts: incident_id. Calls AI classifier (stub first). Updates incident.type and incident.severity. Returns classified record.
GET /incidents	Returns all active incidents ordered by AI priority score DESC. Supports ?lga= filter.
POST /assign	Accepts: incident_id, responder_id. Marks responder busy. Returns: assignment record + ETA (stub: random 5-15 min).
POST /resolve	Marks incident resolved. Frees responder. Logs response_time_minutes.
GET /hospitals/nearby	Accepts: lat, lng, radius_km (default 5). Returns top 5 hospitals sorted by Haversine distance + estimated capacity.
GET /stats/heatmap	Returns incident counts grouped by LGA for last 7 days. Used by analytics panel.
GET /forecast	Returns predicted incident counts for next 6 hours by category. Stub first, model slots in Phase 2.

Testing gate: Before Phase 2 begins, every endpoint must return valid JSON from a curl or Postman call. No broken imports, no 500 errors.

PHASE 2 | AI PIPELINE [HOUR 8 – 20]

RESPONSIBLE: ML/AI ENGINEER (YOU)

This is your zone. Four AI components, built in priority order. Each one can be shipped independently — if time runs short, Components 1 and 2 alone are enough for a strong AI Integration score.

COMPONENT 1 — NLP EMERGENCY CLASSIFIER [HOUR 8 – 12]

What it does: Takes a free-text emergency description and outputs a structured classification: type (medical / fire / security / accident / other), confidence score (0–1), and extracted keywords.

Implementation options (pick one):

- **Option A — Zero-shot (recommended for hackathon speed):** Call Claude or OpenAI API with a structured prompt. Returns JSON with type + confidence. No training required. ~200ms latency.
- **Option B — Fine-tuned sklearn:** Build a TF-IDF + LogisticRegression pipeline on 200 synthetic labelled examples. Trains in under 60 seconds. Fully offline.
- **Option C — HuggingFace zero-shot:** facebook/bart-large-mnli with candidate labels. No API key needed. Heavier dependency but impressive to demo.

Output schema: { type: str, confidence: float, keywords: list[str], raw_input: str }

COMPONENT 2 — SEVERITY SCORER [HOUR 12 – 15]

What it does: Predicts a priority score (1–10) for each incident using structured features. This score drives the dispatch queue order.

Feature set:

- incident_type (one-hot: medical=1.0, fire=0.9, security=0.7, accident=0.8, other=0.3)
- hour_of_day (0–23) — peak hours 6–9 AM and 4–8 PM increase score
- day_of_week (0–6) — weekends have different patterns
- lga_risk_weight (float 0–1) — pre-seeded per LGA based on population density
- citizen_severity_hint (1–5) — self-reported, down-weighted

- keyword_flags (int) — count of high-risk keywords: 'unconscious', 'gun', 'fire'

Model: RandomForestRegressor trained on 500 synthetic incidents. Generates in 10 minutes using a Python script. Serialize with joblib. Slots into /classify endpoint.

COMPONENT 3 — ROUTE OPTIMISER [HOUR 15 – 18]

What it does: Given an incident location and a list of available responders, selects the optimal responder and estimates ETA.

Algorithm:

- Step 1: Haversine distance from each available responder to incident
- Step 2: Apply LGA traffic weight (pre-seeded time-of-day multipliers)
- Step 3: Score = distance_km * traffic_weight / responder_capacity_weight
- Step 4: Return top-1 responder + estimated ETA in minutes

This is deterministic optimisation, not stochastic ML — but it is still AI (operations research / intelligent automation). Frame it correctly in documentation.

COMPONENT 4 — INCIDENT FORECASTER [HOUR 18 – 20]

What it does: Predicts incident counts for the next 6 hours, broken down by type and LGA. Powers the analytics dashboard.

Implementation: Generate 90 days of synthetic hourly incident data with realistic Lagos patterns (morning rush, evening rush, weekend spikes). Fit a simple time-series model — ARIMA or a RandomForest with lag features. Expose predictions via /forecast. Even if predictions aren't perfectly calibrated, the *existence* of a forecasting layer is a strong signal to judges.

PHASE 3 | FRONTEND BUILD [HOUR 10 – 30]

RESPONSIBLE: FRONTEND ENGINEER (RUNS IN PARALLEL WITH PHASE 2)

Three screens. Build them in this order so you always have something demoable even if time runs short.

Screen 1 Citizen Report	Form: emergency type selector (icons), description textarea, auto-detect location button (Geolocation API), severity slider 1–5. Submit calls POST /report then POST /classify. Shows returned priority score and classified type. Build time: 3 hours.
Screen 2 Dispatch Dashboard	Left panel: incident queue sorted by AI priority score, colour-coded by type. Centre: Leaflet map with incident markers + responder pins. Right panel: selected incident detail, hospital recommendations, assign responder button. Real-time poll every 5 seconds. Build time: 6 hours.
Screen 3 Analytics Panel	Row 1: summary cards (active incidents, avg response time, responders on duty). Row 2: Recharts BarChart — incidents by LGA. Row 3: Recharts LineChart — hourly trend. Row 4: forecast table. Build time: 3 hours.

Design system: Navy #0D1B3E, Accent Blue #2E6BE6, White, Alert Red #C0392B, Safe Green #1A7A4A. Roboto font (Google CDN). Consistent 8px grid spacing. This alone moves the UX score.

PHASE 4 | INTEGRATION & TESTING [HOUR 28 – 40]

RESPONSIBLE: FULL TEAM

API integration	Replace all frontend stubs with real API calls. Confirm each screen loads with real seed data. Fix CORS, fix port mismatches.
AI pipeline test	Submit 10 test incidents covering all types and severities. Confirm classifier outputs match expected types. Confirm severity scores rank correctly (unconscious > noise complaint).
Map test	Confirm Leaflet renders Lagos coordinates correctly (6.5°N, 3.4°E). Confirm hospital markers appear within Lagos bounds. Confirm responder pins update after assignment.
Edge cases	Test: empty description, location outside Lagos, all responders busy, hospital list empty. Every edge case returns a graceful error JSON.
Performance	Classify endpoint under 500ms. Dashboard poll under 200ms. No blocking synchronous AI calls in main thread — use background tasks.

PHASE 5 | DOCUMENTATION & SUBMISSION [HOUR 40 – 48]

RESPONSIBLE: FULL TEAM

Documentation is worth 10 points. Teams routinely lose these points by treating it as an afterthought. Do not.

README.md	Project title and tagline. Problem statement (3 sentences). AI integration explanation (list all 4 components). Tech stack. Setup instructions (git clone → pip install → npm install → uvicorn → npm start). Usage guide. Team info.
------------------	---

Architecture diagram	Simple PNG or ASCII diagram showing: Browser → React → FastAPI → AI Pipeline → SQLite. Include in README.
AI explanation section	Dedicated README section: 'How AI Powers LEIN'. Describe each of the 4 AI components, the model used, input features, output format. This is what judges read to score AI Integration.
Demo script	Write a 60-second script: (1) Citizen reports chest pain via mobile. (2) AI classifies: Medical, severity 9/10. (3) Dashboard shows incident at top of queue. (4) Map shows nearest hospital and available ambulance. (5) Dispatcher assigns in one click. (6) Analytics panel shows forecast spike. Rehearse this twice before Demo Day.
Submission form	Complete between 11:59 PM – 12:10 AM on June 8. Have repo URL, team names, and project summary ready in advance. Do not wait until the last minute.

Risk Register

RISK	LIKELIHOOD	MITIGATION
AI API rate limits hit during demo	Medium	Cache last 10 classification results. Demo uses cached responses if live API fails.
Leaflet map doesn't render Lagos	Low	Test coordinates (6.5244, 3.3792) in isolation before integration.
Team member loses internet access	Medium	All dependencies installed locally. App runs fully offline. Use mobile hotspot as backup.
Severity model produces nonsense scores	Low	Hard-code fallback: keyword_flags > 2 → score 8+. Always produces reasonable results.
Submission window missed	Low	Set alarm for 11:45 PM June 7. Form link bookmarked. Repo URL saved in notes.

Frontend-backend CORS error	Medium	Add CORS middleware in FastAPI main.py on Day 1. Test cross-origin calls at
-----------------------------	--------	---

Team Role Assignments

ML/AI Engineer	Phase 0 setup. All of Phase 2 (AI pipeline). AI explanation in README. Demo script. You own the thing that wins this hackathon.
Backend Engineer	Phase 0 setup. All of Phase 1 (API endpoints). Phase 4 integration. Seed data generation.
Frontend Engineer	Phase 0 setup. All of Phase 3 (3 screens). Phase 4 integration. Architecture diagram.
Team Lead / 4th Member	Cross-cutting: documentation, README, submission form, demo rehearsal coordination, risk monitoring. Also assists whichever phase is behind.

The Winning Argument

When a judge asks: 'Why should LEIN win?' — the answer is one sentence: **'LEIN is the only project in this room that could save a life in Lagos tomorrow morning, and we built it in 48 hours with four AI systems working in concert.'**

Last Reminders

- Commit to the repo every 2–3 hours. Judges see your commit history.
- Keep the submission form URL bookmarked. Set a 11:45 PM alarm on June 7.
- If anything breaks — report it immediately in the official community group. Don't wait.
- Seed data is not cheating. Pre-designed architecture is not cheating. Start strong.
- The demo is the product. Practice it until it's automatic.

Built for Lagos. Scalable to every Nigerian city.

NacosCuab Hackathon '26 · June 6 – 8, 2026